Entity Framework Core használata .NET-ben

Code First Fluent API

Tartalomjegyzék

1. Szükséges csomagok telepítése....2
2. 1-N kapcsolat kialakítása....2
2.1 Domain osztályok (Entity) létrehozása....2
2.1.1 A kapcsolat első entitása:....2
2.1.2 A kapcsolat második entitása....3
2.2. Context osztály létrehozása....3
3. M-N kapcsolat létrehozása....4
3.1. Domain osztályok (Entity) létrehozása....4
3.1.1. A kapcsolat első entitása....4
3.1.2. A kapcsolat második entitása....5
3.1.3. A manuálisan létrehozott kapcsolótábla entitás....5
3.2. Context osztály létrehozása....6
4. 1- 1 kapcsolat beállítása....7
4.1 Domain osztályok (Entity) létrehozása....7
4.1.1 A kapcsolat első entitása (Fő entitás)....7
4.1.2 A kapcsolat második entitása (Függő entitás)....7
4.2 Context osztály létrehozása....8
5. Közös beállítások az EF üzembe helyezéséhez....9

1. Szükséges csomagok telepítése (NuGet)

Microsoft.EntityFrameworkCore
Microsoft.EntityFrameworkCore.Design
Microsoft.EntityFrameworkCore.Tools

MySQL (MariaDB) szerver használatához:
MySql.EntityFrameworkCore

MS SQL (Microsoft SQL Server):
Microsoft.EntityFrameworkCore.SqlServer

SQLite:
Microsoft.EntityFrameworkCore.Sqlite

2. 1-N kapcsolat kialakítása

2.1 Domain osztályok (Entity) létrehozása

A példánkban a szerző és a könyvek között alakítunk ki kapcsolatot, úgy, hogy a könyvnek csak egy szerzője lehet (1-N).

2.1.1 A kapcsolat első entitása:

```
namespace BookProjekt.Models
{
   public class Author
   {
      public int Id { get; set; }
      public string Name { get; set; } = string.Empty;
      public DateTime DateOfBirth { get; set; }
      public string PlaceOfBirth { get; set; } = string.Empty;

      // Navigation property
      [JsonIgnore]
      public List<Book>? Books { get; set; }
   }
}
```

- **[JsonIgnore]** - Akkor kell, ha visszaadott JSON-t az osztály alapján kezeljük (nincs DTO), így kerüljük el, hogy ezt is visszaadja a végpont válaszként.

- **public List<Book>? Books { get; set; }** – Navigációs tulajdonság. Azért használunk „List”-et, mert egy szerző több könyvet is írhat a modellünk szerint (1-N). Ez a lista könyv típusokat tárol. A kérdőjel (?) akkor kell, ha a modellünk szerint nem kötelező, hogy egy szerzőnek legyen könyve. Ha kötelező lenne, hogy a szerzőnek mindenképpen legyen könyve, akkor így kell használni:

```
[JsonIgnore]
public List<Book> Books { get; set; } = null!;
```

2.1.2 A kapcsolat második entitása

```
namespace BookProjekt.Models
{
   public class Book
   {
      public int Id { get; set; }
      public string Title { get; set; } = string.Empty;
      public string Category { get; set; } = string.Empty;
      public int PublishedYear { get; set; }
      public int Pages { get; set; }
      public bool Available { get; set; }

      // Foreign Key property
      public int? AuthorId { get; set; }

      // Navigation property
      [JsonIgnore]
      public Author? Author { get; set; }
   }
}
```

- **public int? AuthorId { get; set; }** – Idegen kulcs tulajdonság, a könyvhöz így kapcsoljuk a szerzőt. A kérdőjel (?) azért van benne, mert a modellünk szerint egy könyvnek nem kötelező, hogy legyen szerzője. (Így lehet az SQL táblában a tulajdonság NULL)
  Kötelezőség esetén így kellene használni:

  ```
  public int AuthorId { get; set; }
  ```

- **[JsonIgnore]** - Akkor kell, ha visszaadott JSON-t az osztály alapján adjuk vissza (nincs DTO), így kerüljük el, hogy ezt is visszaadja a végpont.

- **public Author? Author { get; set; }** – Navigációs tulajdonság. Itt nem „List” van, mert a modellünk szerint egy könyvnek csak egy szerzője lehet. A kérdőjel (?) azért van benne, mert a modellünk szerint egy könyvnek nem kötelező, hogy legyen szerzője.
  Kötelezőség esetén így kellene használni:

  ```
  public Author Author { get; set; } = null!;
  ```

2.2. Context osztály létrehozása

```
namespace BookProjekt.Data
{
   public class BookDbContext : DbContext
   {
      public BookDbContext(DbContextOptions<BookDbContext> options) : base(options)
      {
      }
```

```
        public DbSet<Book> Books { get; set; }
        public DbSet<Author> Authors { get; set; }

        // Model konfigurációk
        // Itt konfigurálhatjuk a modellek közötti kapcsolatokat

        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            base.OnModelCreating(modelBuilder);

            modelBuilder.Entity<Book>()
                .HasOne(b => b.Author)
                .WithMany(a => a.Books)
                .HasForeignKey(b => b.AuthorId)
                .IsRequired(false)
                .OnDelete(DeleteBehavior.SetNull);
        }
    }
}
```

- **HasOne(b => b.Author)** - A Book-nak egy Author-a van (lehet null is)
- **WithMany(a => a.Books)** - Az Author-nak több Book-ja lehet
- **HasForeignKey(b => b.AuthorId)** - A külső kulcs a Book.AuthorId
- **IsRequired(false)** - Opcionális kapcsolat (nullable Foreign Key)
- **OnDelete(DeleteBehavior.SetNull)** - Ha törlünk egy szerzőt, a könyveinél az AuthorId null lesz (nem törlődnek a könyvek)

A többi beállítás azonos minden kapcsolat típusnál, ezek a dokumentáció 5. fejezetében találhatóak.

3. M-N kapcsolat létrehozása

A példánkban a szerzők és a könyvek között alakítunk ki kapcsolatot úgy, hogy egy könyvnek több szerzője is lehet, és egy szerzőhöz is tartozhat több könyv (M-N). Mivel a kapcsolatot manuálisan hozzuk létre egy harmadik osztállyal, így később lehetőségünk lesz extra tulajdonságokat (pl. szerkesztés dátuma, szerzői hozzájárulás mértéke) hozzáadni a kapcsolótáblához. Ebben a példában minden könyvnek legalább egy szerzője kell, hogy legyen és minden szerzőnek van legalább egy könyve. (Null nem megengedett!)

3.1. Domain osztályok (Entity) létrehozása

3.1.1. A kapcsolat első entitása

```
namespace BookProjekt.Models
{
    public class Author
    {
        public int Id { get; set; }
        public string Name { get; set; } = string.Empty;
        public DateTime DateOfBirth { get; set; }
        public string PlaceOfBirth { get; set; } = string.Empty;

        // Navigation property a kapcsolótábla felé
        [JsonIgnore]
```

```
        public List<BookAuthor> BookAuthors { get; set; } = new List<BookAuthor>();
    }
}
```

- **[JsonIgnore]** - Akkor kell, ha a visszaadott JSON-t az osztály alapján kezeljük (nincs DTO), így kerüljük el, hogy a végpont válaszként körkörös hivatkozás miatt hibát dobjon, vagy feleslegesen visszaadja ezt a listát.
- **public List<BookAuthor> BookAuthors { get; set; } = new List<BookAuthor>();** – Navigációs tulajdonság. Az M-N kapcsolat miatt a szerző nem közvetlenül a könyvekhez (Book), hanem a köztes kapcsolótáblához (BookAuthor) csatlakozik. A listát érdemes azonnal példányosítani (new List<BookAuthor>()), hogy elkerüljük a NullReferenceException hibákat.

3.1.2. A kapcsolat második entitása

```
namespace BookProjekt.Models
{
    public class Book
    {
        public int Id { get; set; }
        public string Title { get; set; } = string.Empty;
        public string Category { get; set; } = string.Empty;
        public int PublishedYear { get; set; }
        public int Pages { get; set; }
        public bool Available { get; set; }

        // Navigation property a kapcsolótábla felé
        [JsonIgnore]
        public List<BookAuthor> BookAuthors { get; set; } = new List<BookAuthor>();
    }
}
```

- **public List<BookAuthor> BookAuthors { get; set; } = new List<BookAuthor>();** – Navigációs tulajdonság a könyv oldaláról is. Itt is közvetlenül a köztes BookAuthor kapcsolótáblára mutatunk, mivel egy könyvhöz több szerző kapcsolat is tartozhat.

3.1.3. A manuálisan létrehozott kapcsolótábla entitás

```
namespace BookProjekt.Models
{
    public class BookAuthor
    {
        // Önálló elsődleges kulcs
        public int Id { get; set; }

        // Foreign Key tulajdonságok
        public int BookId { get; set; }
        public int AuthorId { get; set; }

        // Navigation tulajdonságok
        [JsonIgnore]
        public Book Book { get; set; } = null!;

        [JsonIgnore]
```

```
        public Author Author { get; set; } = null!;

        // Ide jöhetnek később a plusz tulajdonságok, például:
        // public DateTime AssignedAt { get; set; }
    }
}
```

- **public int Id { get; set; }** – A kapcsolótábla saját, önálló elsődleges kulcsa. Ez egy automatikusan növekvő (Identity) mező lesz az adatbázisban.
- **public int BookId { get; set; } és public int AuthorId { get; set; }** – Idegen kulcs tulajdonságok, amelyek a Book és Author táblákra mutatnak.

3.2. Context osztály létrehozása

```
namespace BookProjekt_Oktatas.Data
{
    public class BookDbContext : DbContext
    {
        public BookDbContext(DbContextOptions<BookDbContext> options) : base(options)
        {
        }

        public DbSet<Book> Books { get; set; }
        public DbSet<Author> Authors { get; set; }
        public DbSet<BookAuthor> BookAuthors { get; set; }

        // Model konfigurációk fluent API-val
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            base.OnModelCreating(modelBuilder);

            // Egyedi index (Unique Constraint) a BookId és AuthorId párosra
            modelBuilder.Entity<BookAuthor>()
                .HasIndex(ba => new { ba.BookId, ba.AuthorId })
                .IsUnique();

            // Book és BookAuthor közötti 1-N kapcsolat leírása
            modelBuilder.Entity<BookAuthor>()
                .HasOne(ba => ba.Book)
                .WithMany(b => b.BookAuthors)
                .HasForeignKey(ba => ba.BookId)
                .OnDelete(DeleteBehavior.Cascade);

            // Author és BookAuthor közötti 1-N kapcsolat leírása
            modelBuilder.Entity<BookAuthor>()
                .HasOne(ba => ba.Author)
                .WithMany(a => a.BookAuthors)
                .HasForeignKey(ba => ba.AuthorId)
                .OnDelete(DeleteBehavior.Cascade);
        }
    }
}
```

- **.HasIndex(ba => new { ba.BookId, ba.AuthorId }).IsUnique()** – Ebben a példában nem használunk összetett kulcsot a szerző és könyv kapcsolótáblájában, így egyedi indexszel

biztosítjuk, hogy a BookId és AuthorId kombinációja egyedi maradjon. Ez megakadályozza, hogy ugyanazt a szerzőt kétszer rendeljük ugyanahhoz a könyvhöz.

A többi beállítás azonos minden kapcsolat típusnál, ezek a dokumentáció 5. fejezetében találhatóak.

4. 1- 1 kapcsolat beállítása

4.1 Domain osztályok (Entity) létrehozása

A példánkban a szerző és a könyvek között alakítunk ki kapcsolatot úgy, hogy a modellünk szerint egy szerzőnek csak egy könyve lehet, és egy könyvhöz is csak egy szerző tartozhat (1-1). (Ez nem életszerű, csak a kapcsolat bemutatása a cél!)

1-1 kapcsolat esetén az Entity Framework Core-nak egyértelműen tudnia kell, hogy melyik a **fő tábla (Principal)** és melyik a **függő tábla (Dependent)**. A függő tárolja majd a külső kulcsot (Foreign Key). A mi példánkban a Book lesz a függő entitás, azaz a könyv hivatkozik a szerzőre.

4.1.1 A kapcsolat első entitása (Fő entitás)

```
namespace BookProjekt.Models
{
    public class Author
    {
        public int Id { get; set; }
        public string Name { get; set; } = string.Empty;
        public DateTime DateOfBirth { get; set; }
        public string PlaceOfBirth { get; set; } = string.Empty;

        // Navigation property
        [JsonIgnore]
        public Book? Book { get; set; }
    }
}
```

- **public Book? Book { get; set; }** – Navigációs tulajdonság. Itt **nem listát** használunk, mert a modell szerint a szerzőhöz legfeljebb egy könyv tartozhat. A kérdőjel (?) azt jelenti, hogy a kapcsolat opcionális: létezhet olyan szerző az adatbázisban, akinek még nincs könyve. Ha kötelezővé szeretnénk tenni, hogy minden szerzőhöz azonnal tartozzon egy könyv, így kellene használni:

```
[JsonIgnore]
public Book Book { get; set; } = null!;
```

4.1.2 A kapcsolat második entitása (Függő entitás)

```
namespace BookProjekt.Models
{
    public class Book
    {
        public int Id { get; set; }
        public string Title { get; set; } = string.Empty;
        public string Category { get; set; } = string.Empty;
        public int PublishedYear { get; set; }
        public int Pages { get; set; }
```

```
        public bool Available { get; set; }

        // Foreign Key property
        public int? AuthorId { get; set; }

        // Navigation property
        [JsonIgnore]
        public Author? Author { get; set; }
    }
}
```

- **public int? AuthorId { get; set; }** – Idegen kulcs tulajdonság. A kérdőjel (?) miatt a kapcsolat opcionális (az SQL táblában NULL értéket is felvehet), vagyis a könyv létezhet szerző nélkül is. Kötelező kapcsolat esetén a kérdőjelet elhagyjuk: **public int AuthorId { get; set; }**
- **public Author? Author { get; set; }** – Navigációs tulajdonság, amely egyetlen szerző objektumra mutat. Kötelező kapcsolat esetén így használnánk:

public Author Author { get; set; } = null!;

4.2 Context osztály létrehozása

```
namespace BookProjekt_Oktatas.Data
{
    public class BookDbContext : DbContext
    {
        public BookDbContext(DbContextOptions<BookDbContext> options) : base(options)
        {
        }

        public DbSet<Book> Books { get; set; }
        public DbSet<Author> Authors { get; set; }

        // Model konfigurációk fluent API-val
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            base.OnModelCreating(modelBuilder);

            modelBuilder.Entity<Book>()
                .HasOne(b => b.Author)
                .WithOne(a => a.Book)
                .HasForeignKey<Book>(b => b.AuthorId)
                .IsRequired(false)
                .OnDelete(DeleteBehavior.SetNull);
        }
    }
}
```

- **.HasOne(b => b.Author)** – A Book-nak egy Author-a van.
- **.WithOne(a => a.Book)** – Az Author-nak is egyetlen Book-ja lehet (ez különbözteti meg az 1-N kapcsolattól, ahol .WithMany() szerepelne).
- **.HasForeignKey<Book>(b => b.AuthorId)** – Megadjuk a külső kulcsot. 1-1 kapcsolatnál a generikus paraméterben (<Book>) kötelező megadni, hogy **melyik táblában** helyezkedik el a fizikai idegen kulcs oszlop (a függő entitásban).

- **.IsRequired(false)** – Opcionális kapcsolat (nullable Foreign Key). Ha kötelezővé tennénk, true értéket kellene kapnia.
- **.OnDelete(DeleteBehavior.SetNull)** – Ha törlünk egy szerzőt, a hozzá tartozó könyvben az AuthorId értéke null lesz, maga a könyv nem törlődik. (Kötelező kapcsolat esetén itt célszerűbb a DeleteBehavior.Cascade használata).

A többi beállítás azonos minden kapcsolat típusnál, ezek a dokumentáció 5. fejezetében találhatóak.

5. Közös beállítások az EF üzembe helyezéséhez

Ebben a fejezetben tárgyalt beállítások szükségesek, hogy az EF-et tudjuk használni, a már előzőleg beállított kapcsolatokhoz.

5.1 appsettings.json fájl beállítása

A ConnectionStrings tulajdonság határozza meg, hogy a használni kívánt adatbázist hogy érjük el. Több DB Context használata esetén több értéket is tartalmazhat.

```
{
 "ConnectionStrings": {
  "BookDbContext": "server=localhost;port=3306;user=root;password=;database=Books"
 },
 "Logging": {
  "LogLevel": {
   "Default": "Information",
   "Microsoft.AspNetCore": "Warning"
  }
 },
 "AllowedHosts": "*"
}
```

A ConnectionStrings példa beállítások különböző adatbázis kezelő rendszerekhez.

MySQL (MariaDB):

```
"ConnectionStrings": {
        "BookDbContext": "server=localhost;port=3306;user=root;password=;database=Books"
},
```

MS SQL (Microsoft SQL Server):

```
"ConnectionStrings": {
    "BookDbContext": "Data Source=.;Initial Catalog=Books; Integrated
    Security=true;TrustServerCertificate=True;"
 },
```

SQLite:

```
"ConnectionStrings": {
        "BookDbContext": "Data Source=Books.db"
},
```

5.2 A Program.cs fájl kiegészítése

A használni kívánt adatbázis kezelő rendszer, illetve a letöltött NuGet csomagok függvényében a beállítások a következőek lehetnek. A kódot a **var app = builder.Build();** kódrészlet felett kell elhelyezni.

MySQL (MariaDB):

```
builder.Services.AddDbContext<BookDbContext>(options => {
    options.UseMySQL(builder.Configuration.GetConnectionString("BookDbContext"));
});
```

MS SQL (Microsoft SQL Server):

```
builder.Services.AddDbContext<BookDbContext>(options => {
    options.UseSqlServer(builder.Configuration.GetConnectionString("BookDbContext"));
});
```

SQLite:

```
builder.Services.AddDbContext<BookDbContext>(options => {
    options.UseSqlite(builder.Configuration.GetConnectionString("BookDbContext"));
});
```

- **builder.Configuration.GetConnectionString("BookDbContext") –** Ebben a részben olvassuk ki a ConnectionStrings beállításból az adatbázis eléréshez szükséges információkat. (A mostani példában BookDbContex a json-ból.) Az adatbázis elérési információt (connection string) átadjuk az adatbázis kezelőnek megfelelő metódusnak (pl: UseMySql).

5.3 Adatbázis létrehozása

Visual Studio-ban a Package Manager Console-ban adjuk ki a következő parancsokat:

Adatbázis migrációhoz szükséges fájlok előállítása:

Add-Migration [Adatbázis migráció megnevezése]

- Itt válasszunk olyan megnevezést, amely röviden leírja a modellen vagy a kapcsolatokon végzett változtatásokat. Az első migrációnál lehet pl.: Init, vagy FirstMigration stb.

Változások kiírása az adatbázisba. Első futtatáskor létrejön az adatbázis és a táblák is.

Update-Database